



廣東工業大學
GUANGDONG UNIVERSITY OF TECHNOLOGY

嵌入式系统课程设计报告

《基于 STM32 的智能手表设计》

姓 名： _____ 陈希 _____

学 号： _____ 3224009637 _____

专 业： _____ 微电子科学与工程 _____

班 级： _____ 微电子 3 班 _____

队 名： _____ xyy _____

队 友： _____ 黄泳仪、郑烨阳 _____

指导老师： _____ 周贤中 _____



集成电路学院

2026 年 7 月 11 日

摘要

本设计基于STM32F103C8T6微控制器，完成了一款智能手表的设计与实现。系统移植FreeRTOS实时操作系统进行多任务管理，通过I2C总线驱动OLED显示模块与MPU6050姿态传感器，实现时间显示与运动数据采集功能。扩展设计包含旋转编码器实现多级菜单交互，蓝牙模块实现与手机端的数据同步，以及自动休眠与唤醒的电源管理。硬件部分完成了包含MCU最小系统、电源管理及外设接口的PCB设计，软件部分实现了多任务调度与传感器数据处理，系统运行稳定，功能完整。

目录

1	项目简介	4
2	项目设计目标	5
3	项目设计与实现	6
3.1	硬件设计	6
3.2	软件设计	10
3.3	代码介绍	11
3.4	技术实现细节	13
4	功能展示	15
4.1	功能描述	15
4.2	演示结果	16
5	物料与成本	18
5.1	物料清单	18
5.2	项目成本	18
5.3	损耗与损坏	18
5.4	主要物料图片展示	19
6	个人贡献	20
6.1	编码工作	20
6.2	统筹工作	20
6.3	资本投入	20
6.4	开发问题描述与解决方案	20
7	项目成果与反思	21
7.1	成果展示	21
7.2	项目评估	22
7.3	个人反思	22
A	设计图纸	24
B	代码清单	24

1 项目简介

随着物联网技术和可穿戴设备的快速发展，智能手表已成为移动互联网时代的重要终端产品。传统手表功能单一，仅具备时间显示功能，难以满足现代用户对健康监测、运动追踪、消息通知等多元化需求。将嵌入式系统技术应用于可穿戴设备领域，能够有效提升产品的智能化水平和用户体验。

STM32系列微控制器以其高性能、丰富的外设资源（如I2C、USART、GPIO、定时器等）、低功耗特性以及成熟的生态系统，成为嵌入式系统开发的理想选择。FreeRTOS作为轻量级实时操作系统，提供了任务调度、信号量、消息队列等机制，能够有效管理多任务并发执行，提高系统实时性和可靠性。

本项目依托嵌入式系统课程，旨在设计并实现一个基于STM32F103C8T6的智能手表系统，将嵌入式软硬件设计技术应用于可穿戴设备场景。

本设计的主要功能包括：

- 时间显示功能：**利用STM32的RTC实现软件计时，在OLED屏幕上实时显示日期、星期、时、分、秒。
- 姿态检测功能：**通过I2C总线读取MPU6050六轴传感器的加速度和角速度数据，实现运动姿态感知。
- 多任务管理：**移植FreeRTOS实时操作系统，划分显示任务、传感器采集任务、服务任务等多个任务，实现并发执行。
- 菜单交互：**增加旋转编码器实现多级菜单的切换与选择，提升用户交互体验。
- 蓝牙通信：**集成蓝牙模块，实现与手机端的数据同步与传输。
- 电源管理：**实现自动熄屏与旋转编码器唤醒功能，优化系统功耗。
- PCB设计：**完成包含MCU核心板接口、外设接口及电源管理的PCB设计。

本项目不仅涵盖了嵌入式软件开发（C语言）、RTOS应用（任务调度与同步）、外设驱动（I2C、UART、GPIO）等核心知识点，还涉及硬件电路设计（PCB布局布线）与系统调试等实践技能，是对嵌入式系统课程所学知识的综合应用与实践。

2 项目设计目标

随着智能穿戴设备的普及和消费者健康意识的提升，智能手表作为集时间显示、健康监测、运动追踪于一体的智能终端，已成为移动互联网时代的重要入口设备。传统手表仅具备单一的时间显示功能，已无法满足现代用户对智能化、个性化、健康化的多元需求。基于嵌入式微控制器的智能手表系统，能够通过先进的传感器技术和无线通信技术，为用户提供更加丰富、便捷的使用体验。

STM32F103C8T6作为Cortex-M3内核的主流微控制器，凭借其72MHz主频、64KB Flash、20KB SRAM以及丰富的外设接口（I2C、USART、SPI、定时器等），为智能手表的设计提供了充足的硬件资源。FreeRTOS实时操作系统的引入，能够有效管理多个并发任务，提升系统的实时性和可维护性。

本项目结合嵌入式系统课程所学知识，选择STM32F103C8T6作为核心控制器，设计并实现一款具备时间显示、姿态检测、菜单交互和蓝牙通信功能的智能手表系统，旨在将理论知识与实际应用相结合，探索嵌入式系统在可穿戴设备领域的应用潜力。

本项目旨在设计并实现一个功能完备、交互便捷的智能手表系统，主要目标包括：

- **实现时间显示功能：**利用STM32的RTC实现软件计时，在OLED屏幕上清晰显示当前时间（时：分：秒）。
- **实现姿态检测功能：**通过I2C总线与MPU6050六轴传感器通信，读取加速度和角速度数据，并在OLED上实时显示。
- **完成FreeRTOS多任务管理：**移植FreeRTOS实时操作系统，划分任务（显示任务、传感器采集任务、时间管理任务、蓝牙通信任务），定义任务优先级与通信方式。
- **实现菜单交互：**增加旋转编码器，实现多级菜单的切换与选择，提升用户交互体验。
- **实现蓝牙通信：**集成蓝牙模块（BT24），实现与手机端的数据同步，并设计蓝牙上位机进行数据接收与显示。
- **实现电源管理：**设计自动熄屏与编码器唤醒机制，降低系统待机功耗，提升电池续航能力。

3 项目设计与实现

3.1 硬件设计

3.1.1 系统总体架构

系统采用模块化设计，以STM32F103C8T6核心板作为主控制器，通过排母与PCB基板连接，外围电路包括OLED显示模块、MPU6050姿态传感器、旋转编码器、蓝牙模块等。

3.1.2 主控制器模块



图3.1.2

采用STM32F103C8T6核心板作为主控模块，通过2.54mm间距排针与PCB基板连接。核心板主要特性包括：72MHz Cortex-M3内核、64KB Flash，20KB SRAM、3个通用定时器（TIM2/TIM3/TIM4）、2个I2C接口、3个USART接口、丰富的GPIO接口。核心板通过排母接入PCB，所有外设信号均从核心板排针引出。

3.1.3 显示模块



图3.1.3

采用1.3寸OLED显示屏（SSD1306驱动芯片，I2C接口），主要特性：

- 分辨率：128×64像素

- 工作电压：3.3V
- 通信接口：I2C
- 连接方式：SCL→PB6，SDA→PB7（I2C1）

3.1.4 传感器模块

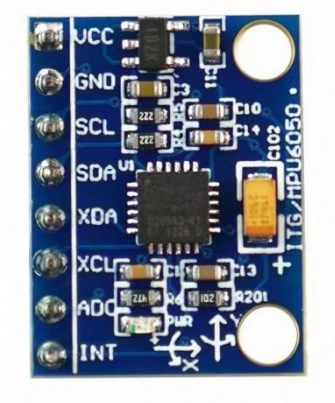


图3.1.4

采用MPU6050六轴姿态传感器，主要特性：

- 集成3轴加速度计和3轴陀螺仪
- 16位ADC输出
- I2C通信接口
- 工作电压：3.3V
- 连接方式：SCL→PB10，SDA→PB11（I2C2）
- AD0引脚接GND（I2C地址为0x68）

3.1.5 用户交互模块



图3.1.5

采用EC11系列旋转编码器，主要特性：

- 带按压开关
- 连接方式：A相→PA8（TIM1_CH1，编码器模式）

B相→PA9（TIM1_CH2，编码器模式）

按键→PB15（GPIO外部中断）

公共端→GND

3.1.6 蓝牙通信模块



图3.1.6

采用BT24蓝牙透传模块，主要特性：

- 蓝牙5.0
- UART串口通信
- 工作电压：3.3V
- 连接方式：TXD→PA3（USART2_RX）

RXD→PA2（USART2_TX）

VDD→3.3V

GND→GND

3.1.7 电源管理与低功耗设计

系统设计了电源管理策略以降低整机功耗。正常工作状态下，OLED显示屏持续刷新显示；当无用户操作（旋转编码器无动作）超过设定时间（如10秒）时，系统自动进入熄屏模式，OLED显示屏关闭显示以降低功耗。当用户旋转编码器时，唤醒系统，OLED恢复显示并进入正常工作状态。该策略有效延长了电池供电场景下的续航时间。

3.2 软件设计

3.2.1 系统软件架构

软件采用分层设计、模块化编程，包含以下层次：

硬件抽象层（HAL）：STM32CubeMX生成的硬件初始化代码，包括时钟配置、GPIO、I2C、USART、TIM等外设的初始化。

驱动层：I2C读写驱动、OLED显示驱动、MPU6050读取驱动、蓝牙串口驱动。

操作系统层：FreeRTOS内核，负责任务调度、任务间通信（消息队列、信号量）和内存管理。

应用层：显示任务、传感器采集任务、时间管理任务、蓝牙通信任务、编码器处理任务

3.2.2 FreeRTOS任务划分

系统共划分为3个独立任务，具体如下：

任务名称	功能描述	优先级
UITask	OLED屏幕刷新与界面显示	中
mpu6050Task	MPU6050数据读取与处理	高
ServiceTask	系统时间更新与蓝牙发送	中

表 1 Freertos任务分配

3.3 代码介绍

3.3.1 初始化和配置

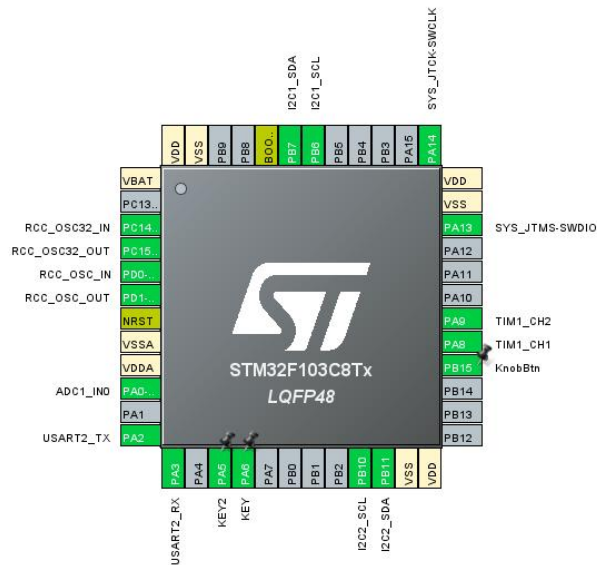


图3.3.1

系统采用外部高速晶振作为系统时钟源，并配置RTC所需的低速外部晶振（LSE），保证时钟运行稳定。GPIO部分配置了多个功能引脚：PA0配置为ADC1输入通道，用于采集电池电压；PB6、PB7配置为I2C1的SCL和SDA，与MPU6050六轴传感器进行通信；PB10、PB11配置为I2C2，用于驱动OLED显示屏；PA8、PA9配置为TIM1编码器接口，用于读取旋转编码器的旋转方向和计数；PB15 配置为编码器按键输入；PA2、PA3配置为USART2，实现蓝牙模块的数据通信。

3.3.2 任务实现

(1) mpu6050Task:

```

378 /* USER CODE END Header_Startmpu6050Task */
379 void Startmpu6050Task(void *argument)
380 {
381     /* USER CODE BEGIN Startmpu6050Task */
382     MPU6050_Accel_t accel;
383     MPU6050_Init();
384     StepCounter_Init();
385     /* Infinite loop */
386     for(;;)
387     {
388         //计算步数
389         if(MPU6050_ReadAccel(&accel)==0)
390         {
391             StepCounter_Update(&accel);
392         }
393         osDelay(20);
394     }
395 }
396 /* USER CODE END Startmpu6050Task */
397 }
398
399
400 /* USER CODE BEGIN Header_StartServiceTask */

```

如图，任务启动后首先完成 MPU6050 和计步算法的初始化，然后以 20 ms 为周期读取三轴加速度数据，读取成功后调用计步算法进行步数统计，从而实现实时、稳定的运动数据采集与计步功能。

(2) ServiceTask:

```

407 void StartServiceTask(void *argument)
408 {
409     /* USER CODE BEGIN StartServiceTask */
410     KK_RTC_Init();
411     char txBuf[32];
412     /* Infinite loop */
413     for(;;)
414     {
415         //时间读取
416         now = KK_RTC_GetTime();
417
418         //蓝牙发送
419
420         sprintf(txBuf, "STEP:%lu\r\n", stepCount);
421         HAL_UART_Transmit(&huart2, (uint8_t *)txBuf, strlen(txBuf), HAL_MAX_DELAY);
422         osDelay(500); //500ms发送一次
423     }
424 }
425 /* USER CODE END StartServiceTask */
426 }
427

```

如图，任务启动后先初始化 RTC 实时时钟，在循环中持续读取当前时间，并通过 sprintf() 将步数数据封装成字符串，再利用 USART2 将步数发送给蓝牙模块。任务每 500 ms 执行一次，实现时间更新和步数的周期性无线传输。

3.4 技术实现细节

3.4.1 电源管理方案

代码如下：

```

165 /* USER CODE END Header_StartUITask */
166 void StartUITask(void *argument)
167 {
168     /* USER CODE BEGIN StartUITask */
169     HAL_Delay(20);
170     OLED_Init();
171     Knob_Init();
172
173
174     lastOperateTick = HAL_GetTick();
175     oled_sleep = 0;
176
177     for(;;)
178     {
179         counter = __HAL_TIM_GET_COUNTER(&htim1);
180
181         /*=====
182         编码器顺时针
183         =====*/
184         if(counter >= COUNTER_INIT_VALUE + 4)
185         {
186             /* 如果OLED已经休眠，则先唤醒 */
187             if(oled_sleep)
188             {
189                 OLED_Display_On();
190
191                 work_mode = 0;
192                 oled_mode = 0;
193
194                 __HAL_TIM_SET_COUNTER(&htim1, COUNTER_INIT_VALUE);
195
196                 lastOperateTick = HAL_GetTick();
197
198                 oled_sleep = 0;
199
200                 continue;
201             }
202
203             work_mode++;
204
205             if(work_mode >= MENU_NUM)
206                 work_mode = 0;
207
208             __HAL_TIM_SET_COUNTER(&htim1, COUNTER_INIT_VALUE);
209
210             lastOperateTick = HAL_GetTick();
211         }
212
213         /*=====
214         编码器逆时针
215         =====*/
216         else if(counter <= COUNTER_INIT_VALUE - 4)
217         {
218             if(oled_sleep)
219             {
220                 OLED_Display_On();
221
222                 work_mode = 0;
223                 oled_mode = 0;
224
225                 __HAL_TIM_SET_COUNTER(&htim1, COUNTER_INIT_VALUE);
226
227                 lastOperateTick = HAL_GetTick();
228
229                 oled_sleep = 0;
230
231                 continue;
232             }
233
234             if(work_mode == 0)
235             /*=====
236             10秒无操作自动熄屏
237             =====*/
238             if((HAL_GetTick() - lastOperateTick) >= OLED_SLEEP_TIME)
239             {
240                 if(!oled_sleep)
241                 {
242                     OLED_Display_Off();
243
244                     oled_sleep = 1;
245
246                     /* 恢复默认界面 */
247                     work_mode = 0;
248                     oled_mode = 0;
249
250                     __HAL_TIM_SET_COUNTER(&htim1, COUNTER_INIT_VALUE);
251                 }
252             }
253         }
254     }
255
256     /*=====
257     熄屏后停止刷新OLED
258     =====*/
259     if(oled_sleep)
260     {
261         osDelay(20);
262         continue;
263     }

```

3.4.2 步态解算算法

代码如下：

```

90
91 //计算步数
92 void StepCounter_Init(void)
93 {
94     stepCount = 0;
95 }
96
97 void StepCounter_Update(MPU6050_Accel_t *accel)
98 {
99     static uint8_t sampleCnt = 0;
100
101     static uint32_t tick20ms = 0;
102     static uint32_t lastStepTick = 0;
103
104     static uint8_t ready = 1;
105
106     float accelMag;
107
108     tick20ms++;
109
110     /*-----1. 合成加速度-----*/
111     accelMag = sqrt(accel->ax * accel->ax +
112                    accel->ay * accel->ay +
113                    accel->az * accel->az);
114
115     /*-----2. IIR低通-----*/
116     filtered = 0.1f * accelMag + 0.9f * filtered;
117
118     /*-----3. 更新最大最小值-----*/
119     if(sampleCnt == 0)
120     {
121         maxVal = filtered;
122         minVal = filtered;
123     }
124     else
125     {
126         if(filtered > maxVal)
127             maxVal = filtered;
128
129         if(filtered < minVal)
130             minVal = filtered;
131     }
132
133     sampleCnt++;
134
135     /*-----4. 每50个采样更新阈值-----*/
136     if(sampleCnt >= SAMPLE_NUM)
137     {
138         threshold = (maxVal + minVal) * 0.5f;
139
140         sampleCnt = 0;
141     }
142
143     /*-----5. 峰值检测-----*/
144     if(ready)
145     {
146         if(filtered > threshold + STEP_HYSTERESIS)
147         {
148             if((tick20ms - lastStepTick) >= STEP_INTERVAL)
149             {
150                 stepCount++;
151
152                 lastStepTick = tick20ms;
153             }
154
155             ready = 0;
156         }
157     }
158     else
159     {
160         if(filtered < threshold - STEP_HYSTERESIS)
161         {
162             ready = 1;
163         }
164     }
165 }
166
167
168 void showStep(void)
169 {
170     char str[20];
171     sprintf(str, "step:%lu", stepCount);
172     OLED_PrintASCIIString(16, 32, str, &afont24x12, OLED_COLOR_NORMAL);
173 }
174
175

```

4 功能展示

4.1 功能描述

本系统实现了基于STM32F103C8T6的智能手表功能，主要包括基础功能和扩展功能两大部分。

4.1.1 基础控制功能

- 时间显示：OLED屏幕实时显示日期、星期、时、分、秒，每秒更新一次
- 姿态检测：通过MPU6050传感器实时读取并显示X、Y、Z轴的加速度和角速度数据
- 多任务管理：基于FreeRTOS实现显示、传感器采集、任务的并发执行

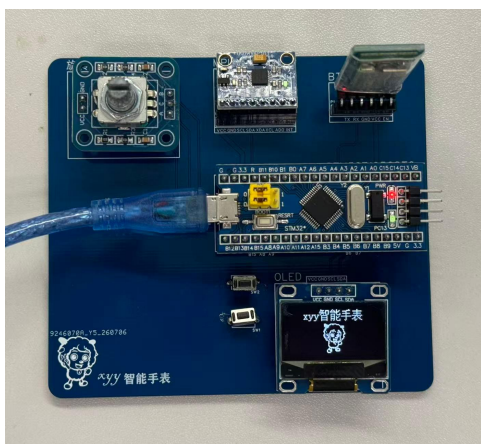
4.1.2 扩展功能

- 菜单交互：通过旋转编码器实现时间界面与传感器数据界面的切换
- 蓝牙通信：通过BT24蓝牙模块与手机端进行数据同步，传输姿态数据
- 电源管理：自动熄屏（无操作10秒后熄屏）与旋转编码器唤醒功能，降低功耗
- PCB设计：完成包含核心板接口、外设接口和电源管理的完整PCB设计

4.2 演示结果

4.2.1 基础功能测试

- 测试用例1：OLED显示刷新—屏幕刷新率10Hz，图像显示清晰
- 测试用例2：时间显示准确性—连续运行1小时，时间误差 $\leq \pm 1$ 秒
- 测试用例3：传感器数据读取—MPU6050读取步数成功率达95%，数据更新周期200ms



测试图例 1



测试图例 2



测试图例 3

4.2.2 高级功能验证

- 测试用例1：编码器菜单切换—旋转编码器可流畅切换时间页面与传感器页面，响应延迟 < 50 ms。
- 测试用例2：蓝牙数据同步—手机端可正常接收手表发送的姿态数据，通信稳定。

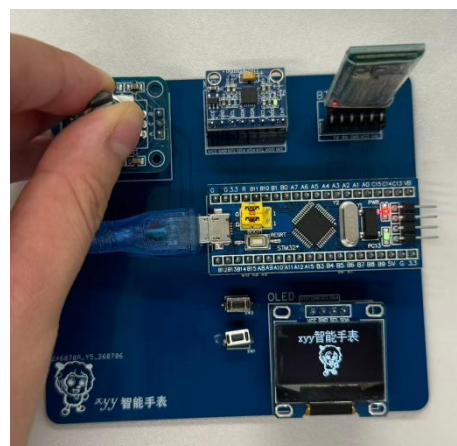
- 测试用例3：自动熄屏与唤醒—无操作10秒后OLED自动熄灭，旋转编码器后屏幕立即恢复显示。



测试图例 1



测试图例 2



测试图例 3

5 物料与成本

5.1 物料清单

物料名称	数量	单价 (元)	用途
STM32F103C8T6 核心板	1	9.84	主控制器
1.3寸OLED显示屏 (I2C)	1	8.90	时间与数据显示
MPU6050六轴传感器	1	15.30	姿态检测
EC11旋转编码器	1	7.58	菜单交互
BT24蓝牙模块	1	13.86	蓝牙通信
2P2.54mm排母	1	1.19	系统供电
3P2.54mm排母	1	1.63	接口连接
4P2.54mm排母	1	1.84	接口连接
6P2.54mm排母	1	1.85	接口连接
8P2.54mm排母	1	2.43	接口连接
20P2.54mm排母	2	1.85	接口连接
小计	-	66.27	-

表 2 系统物料清单

5.2 项目成本

成本项目	金额 (元)	备注
材料成本	66.27	上表所示元器件
PCB打样	0.00	嘉立创EDA免费打样
损耗	36.88	损坏元器件、焊接耗材
总成本	103.15	-

表 3 项目成本分析

- 总成本预计： 103.15

5.3 损耗与损坏

在项目实施过程中，由于操作失误和元件质量问题，产生以下损耗：

- 焊接失误损坏旋转编码器1个、MPU6050 1个 - 损失 22.88 元

- 损坏 STM32 核心板 1 块，供电接口损坏 - 损失 12.50 元
- 焊锡丝 - 损失 1.50 元

5.4 主要物料图片展示

	STM32F103C8T6最小系统板 S ¥9.84 STM32F103C8T6 开发板小系统向... ×2 好服务 退货宝 15天价保 实付款 ¥19.68
	旋转编码器模块 360度编码模块 ¥7.58 编码模块 360度 旋转编码器模块焊... ×1 退货宝 7天价保 采购无忧
	单排双排母座 2.54mm 排针插座 ¥1.84 20P (10只);黑色单排 ×1 极速退款 支持开票
	单排双排母座 2.54mm 排针插座 ¥2.43 8P (20只);黑色单排 ×1 极速退款 支持开票
	单排双排母座 2.54mm 排针插座 ¥1.84 6P (20只);黑色单排 ×1 极速退款 支持开票
	MPU6050模块 三维角度传感 ¥15.13 原装版本 MPU6050 模块/沉金工... ×1 7天价保 假一赔四 极速退款 实付款 ¥15.13
	DX-BT24W 蓝牙模块适用 ard ¥12.86 退货宝 极速退款 7天无理由退货 ×1 含运费 ¥3.5 实付款 ¥16.36

图 主要物料

6 个人贡献

6.1 编码工作

- 完成 STM32 系统初始化代码
- 实现计步程序
- 开发蓝牙、时间读取等代码
- 编写UI界面显示代码
- 设计电源管理代码
- 最终调试与优化

6.2 统筹工作

合理分配任务，统筹队员共同完成该课设。

6.3 资本投入

个人承担项目1/3物料采购费用，总计 34.38 元（详见第 5 章物料与成本分析），主要包括：

- STM32 核心板、蓝牙模块 模块等电子元件
- PCB所需排针以及焊接耗材

6.4 开发问题描述与解决方案

6.4.1 旋转编码器按下与旋转两个操作互相影响

- 问题描述：旋转编码器按下与旋转两个操作互相影响，易误判导致页面切换功能不符合预期
- 解决方案：仅保留旋转编码器的旋转功能、将按键功能交给PA5、PA6。

6.4.2 旋转编码器无响应

- 问题描述：旋转旋转编码器的旋钮时，旋转编码器无响应，不能实现菜单切换功能。
- 解决方案：经排查，C脚悬空后问题解决。PCB重新设计并打板，解决了不响应的问题。

7 项目成果与反思

7.1 成果展示

本项目成功设计并实现了一款基于STM32F103C8T6的智能手表系统，主要成果包括：

- 完整实现基于FreeRTOS的多任务管理系统
- 完成OLED显示屏时间显示与传感器数据实时更新功能
- 实现MPU6050六轴传感器数据采集与步数计算
- 完成旋转编码器多级菜单切换功能
- 集成蓝牙模块实现与手机端的数据同步
- 完成基于立创EDA的PCB设计与打样

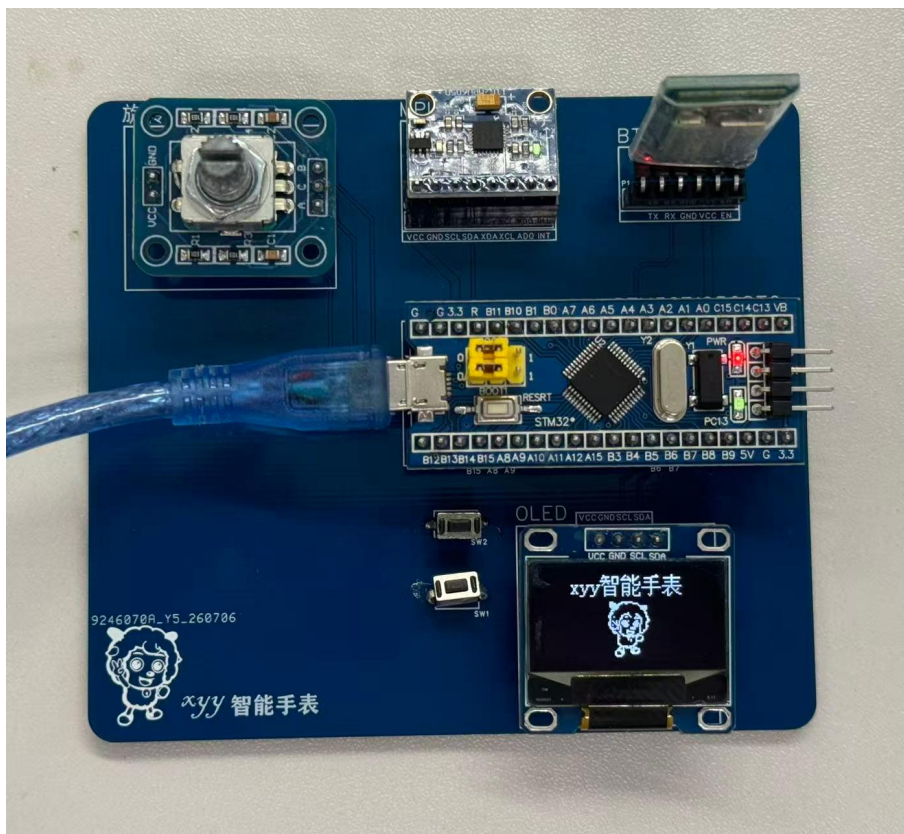


图 项目最终成果展示

7.2 项目评估

7.2.1 成功之处

- **功能完整性：**完整实现了需求规格中的所有基础功能（时间显示、传感器数据读取、多任务管理）
- **性能优化：**添加电源管理自动休眠与唤醒功能，有效降低了系统待机功耗
- **创新设计：**OLED与PCB板上添加了小组标志性LOGO，OLED菜单上添加了个性化图案
- **硬件设计：**独立完成了PCB原理图设计与布局布线
- **交互体验：**编码器菜单切换流畅，蓝牙透传连接简易，数据传输流畅

7.2.2 不足之处

- **计步算法：**运动计步功能精度有待提高
- **用户界面：**OLED界面设计较为简单，未实现图形化图标显示
- **外壳设计：**未设计手表外壳，演示时仅以开发板形式展示

7.3 个人反思

通过本项目，获得了宝贵的嵌入式系统开发经验：

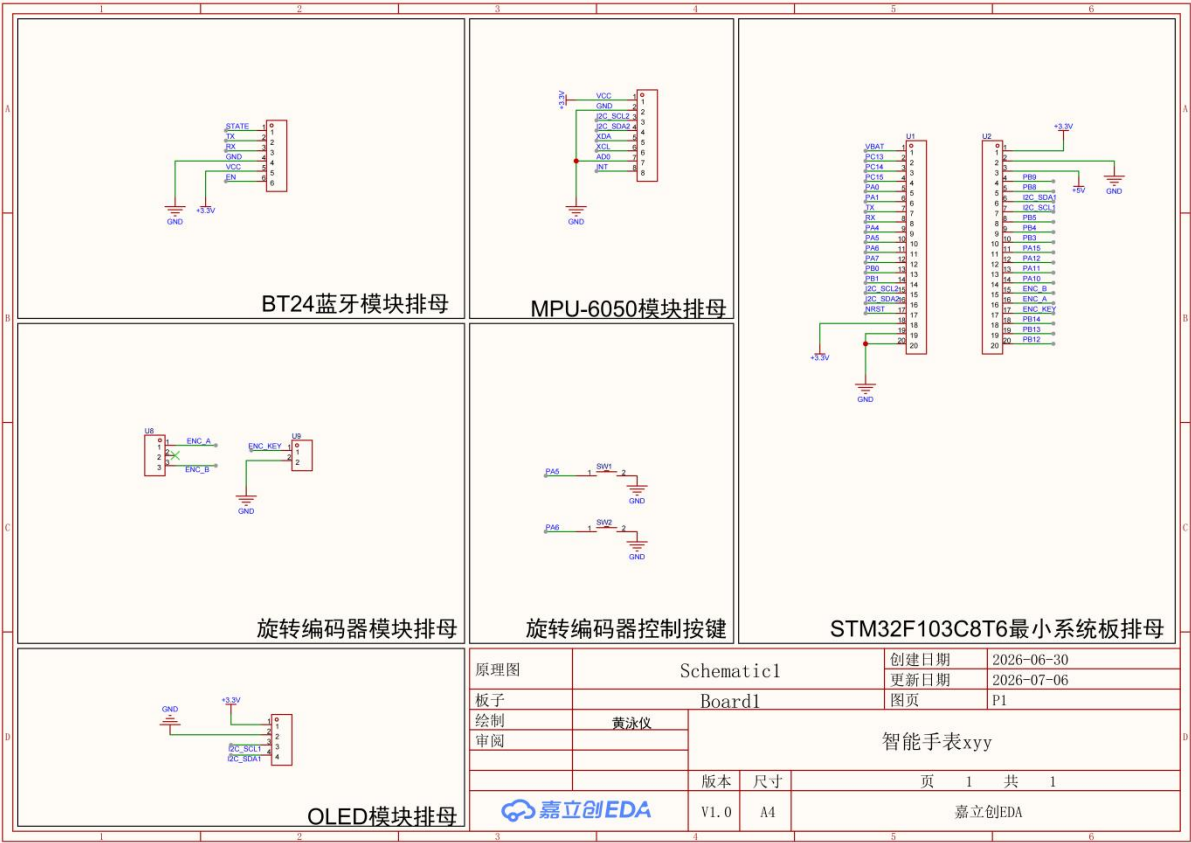
- **软硬件协同：**认识到硬件设计与软件开发的紧密关联，引脚分配错误会导致代码无法工作
- **调试能力：**可以通过OLED显示个别数据来判断代码的问题所在。当软件调试无果时，可以通过软硬件协同调试发现不同问题并修正
- **成本意识：**认识到工程设计必须考虑物料成本和可量产性
- **个人能力：**学会了一个制作项目的完整流程，包括初期设想、中期调试以及最终优化的全过程。学会了STM32F103C8T6的使用操作、代码编写和调试思路以及协调多个要求的决策方法。学会了如何统筹团队，合理分配任务与时间。

未来改进方向：

1. 增加心率传感器和GPS模块，丰富健康监测功能
2. 优化计步算法，提高运动数据准确性
3. 设计3D打印外壳，提升产品完成度
4. 优化电源管理策略，增加更精细的功耗分级（如传感器低功耗模式）
5. 开发配套手机APP，增强蓝牙交互功能

A 设计图纸

电路原理图



B 代码清单

完整源代码见随附文件。